JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202219903 이름: 박예찬 제출일: 2026-10-28

서비스 이름 게임 공략 게시판

기술 조합과 버전 Vue 3.5.43 (Vite 8.3.4) / Express 5.3.0 (jsonwebtoken 9.0.3, sql.js 1.14.2) / Node.js v24.15.0, macOS 15.6.1

GitHub 저장소 URL https://github.com/ParkYeChan2979/game-guide-board

소스 코드 기준 main 브랜치 / 코드 구현 기준 커밋 e67b522

게임을 하다가 막힌 플레이어가 보스 공략, 아이템 위치, 빌드 추천 같은 정보를 찾아보고 공유하는 게시판입니다. 공략은 로그인하지 않아도 누구나 목록과 상세를 읽을 수 있고, 공략 작성은 로그인한 사용자만 할 수 있습니다. 다른 사람이 내 공략을 고치거나 지우지 못하도록 작성자 본인만 수정·삭제할 수 있게 했습니다.

기술 조합은 Vue + Express 를 선택했습니다. 프론트와 백엔드를 모두 JavaScript 로 작성할 수 있어 언어를 바꾸지 않고 화면에서 서버까지 요청 흐름을 따라갈 수 있고, 공지에서 수업 코드를 이어 쓰기 좋은 조합으로 안내되었기 때문입니다.

제공 템플릿에서는 인증 부분(backend-express/src/auth.js 전체, frontend-vue/src/api.js 의 FE-01~04)과 화면의 안내 문구·오류 처리(App.vue)를 변경했습니다. 라우트, 입력 검사, SQLite 저장소, 테스트 계정 코드는 제공 코드를 그대로 사용했고, 폴더 이름과 구조도 템플릿과 같게 유지했습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	app.js GET 라우트, App.vue	제공 코드 유지, 비로그인 조회 확인(T01)
로그인과 JWT 발급	완료	auth.js issueToken, api.js login	HS256, 기본 TTL 90 초
인증과 내 정보 조회	완료	auth.js currentUserId, GET /api/me	토큰 없음·변조·만료를 401 code 로 구분
본인 글 작성·수정·삭제	완료	auth.js assertOwner, api.js canEdit	작성자는 토큰으로 결정, 타인 변경 403
만료·로그아웃·오류 표시	완료	api.js request, App.vue run·logout	401 이면 로그인 해제, 403 은 로그인 유지
DB 데이터 유지	완료	store.js(제공), data/board.sqlite	서버 재시작 후 유지(T15), 로그인은 새로고침 시 해제

구조와 변경 내역

```
브라우저  http://127.0.0.1:4173  (Vue 빌드본)
  App.vue ──▶ api.js   session = { accessToken, user }   ← 토큰은 메모리에만 보관
                │  fetch('/api/...')  + Authorization: Bearer <JWT>  (토큰이 있을 때만)
                ▼
Vite preview 프록시   /api  →  http://127.0.0.1:8000
                ▼
Express  app.js 라우트 ──▶ auth.js   issueToken / currentUserId / assertOwner
                ▼
store.js (sql.js) ──▶ backend-express/data/board.sqlite
```

로그인 흐름 로그인 폼 → api.js login() → POST /api/auth/login (토큰 없음) → app.js 가 fields()로 입력 검사, verifyCredentials()로 비밀번호 확인 → auth.js issueToken()이 JWT 발급 → 200 {accessToken, expiresIn, user} → api.js 가 session 변수에 저장하고 화면에 'Alice 님 로그인 중'을 표시합니다.

게시글 수정 흐름 수정 저장 → api.js update() → PUT /api/posts/{id} + Authorization: Bearer <JWT> → currentUserId()로 토큰 검증(실패 401) → postId()·getPost()로 ID 와 글 확인(400·404) → assertOwner()로 작성자 비교(실패 403) → fields()로 입력 검사(400) → store.update()로 SQLite 에 저장 → 200 수정된 글.

토큰 저장과 전송 토큰은 api.js 의 session 변수(이 탭의 메모리)에만 있고 localStorage 에는 저장하지 않습니다. 로그인 상태에서는 모든 /api 요청에 Authorization 헤더가 붙지만, 서버는 GET /api/me 와 POST·PUT·DELETE /api/posts 에서만 검사하고 공개 조회는 토큰 없이도 200 입니다(T01). 로그아웃, 401 응답, 새로고침 시 토큰이 사라집니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-express/src/auth.js / issueToken	시작 코드: 501 NOT_IMPLEMENTED 오류만 반환	HS256 JWT 발급(sub·iss·aud·exp, TTL)	BE-01 로그인 성공 시 토큰 발급
auth.js / currentUserId	시작 코드: 토큰 검사 없이 데모 사용자 ID 1 반환	Bearer 추출, jwt.verify(HS256·iss·aud·만료), exp·sub 확인, 계정 확인, 401 code 구분	BE-02 보호 API 인증
auth.js / assertOwner	시작 코드: 아무 검사도 하지 않음	검증된 사용자 ID ≠ 글 authorId 이면 403 FORBIDDEN	BE-03 작성자 권한
auth.js / 모듈 시작부 설정 검사	시작 코드: 없음(새로 추가)	JWT_SECRET 64 자리 16 진수·예시 키·TOKEN_TTL 검사, 실패 시 서버 시작 거부	빈 키·예시 키 거부 요구사항
backend-express/src/app.js / app.listen 로그	시작 코드: 서버 시작 시 '인증 TODO 미완성' 출력	'게임 공략 게시판 API' 문구로 교체	완성본 표시
frontend-vue/src/api.js / login, authorizationHeaders	시작 코드: login 은 '학생 구현 항목' 안내 오류만 발생, authorizationHeaders 는 빈 객체 반환	토큰·사용자 메모리 저장, 토큰이 있을 때만 Bearer 헤더	FE-01·FE-02

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
api.js / request, canEdit	시작 코드: 오류 메시지만 표시(401 처리 없음), canEdit 은 항상 true	401 세션 해제, code 별 안내, 204 파싱 생략, 작성자 비교	FE-03·FE-04
frontend-vue/src/App.vue / run, save, logout, 화면 문구	시작 코드: 인증 미완성 데모 배너, 비로그인 글쓰기 가능	비로그인 글쓰기 안내, 404 시 목록 갱신, 로그아웃 시 폼 초기화, 게임 공략 문구	FE-03·FE-04, 주제 반영
style.css .message.error, index.html title	시작 코드: 기본 메시지 스타일, '우리의 기록' 제목	오류 안내 빨간색 표시, 페이지 제목 변경	오류와 정상 안내 구분

작성 출처: 위 표의 변경은 모두 AI(Claude Code)가 생성한 코드를 반영했고, 본인이 코드를 읽어 검토한 뒤 T01~T17 테스트로 동작을 검증했습니다. app.js 의 라우트·입력 검사(fields, postId)·오류 응답, store.js, users.js, errors.js, vite.config.js 는 제공 코드를 사용했습니다. 보조 파일로 docs/AI_활용기록.md 와 docs/evidence/테스트기록.md 를 추가했습니다.

로컬 실행 기록

운영체제와 런타임 macOS 15.6.1, Node.js v24.15.0, Python 3.10.2(검사 도구), Chrome 154

설치 backend-express 와 frontend-vue 에서 각각 npm ci

환경설정 backend-express/.env.example 을 .env 로 복사하고 JWT_SECRET 을 crypto.randomBytes(32)로 만든 64 자리 16 진수로 설정(키 값은 기록하지 않음), TOKEN_TTL 기본 90 초

프론트 빌드 frontend-vue 에서 npm run build → 성공(built in 76ms) 후 npm run preview

백엔드 실행 backend-express 에서 npm start → http://127.0.0.1:8000

SQLite 파일 경로 backend-express/data/board.sqlite (서버 시작 시 자동 생성, 커밋 제외)

4173 번 접속과 API 연결 http://127.0.0.1:4173 에서 게시판이 열리고 Network 에서 GET /api/posts 200 을 확인했습니다(E02_preview.png).

서버 재시작 후 데이터 유지 Ctrl+C 로 백엔드 종료 → 연결 실패 확인 → npm start 재시작 → 글 20 의 id·제목·내용·작성자·createdAt 이 동일했습니다(T15).

새 폴더 clone 재현 ~/Desktop/clone-test 에 GitHub 저장소를 clone 해 README 절차(npm ci, .env 복사와 새 키 설정, npm start, npm run build·preview)대로 실행했고, 빈 DB 에서 자동 시작해 첫 글이 201, Location /api/posts/1 로 등록되는 것을 확인했습니다(C01~C04).

증거 위치 docs/evidence/E01_build.png, E02_preview.png, T15_restart_*.png, C01~C05(새 폴더 재현) / 자세한 실행 방법은 저장소 README.md

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-express/src/app.js / POST /api/auth/login 라우트, backend-express/src/auth.js / issueToken

코드 발췌

```
// backend-express/src/app.js  (POST /api/auth/login)
46    const input = fields(req.body, { email: 254, password: 128 });
47    const user = verifyCredentials(input.email, input.password);
48    if (!user)
49      throw new ApiError(
50        401,
51        "INVALID_CREDENTIALS",
52        "이메일 또는 비밀번호를 확인하세요.",
53      );
54    const accessToken = issueToken(user);
// backend-express/src/auth.js
27  export function issueToken(user) {
28    return jwt.sign({}, SECRET, {
29      algorithm: "HS256",
30      subject: String(user.id),
31      issuer: ISSUER,
32      audience: AUDIENCE,
33      expiresIn: TTL,
34    });
35  }
```

동작과 응답 코드 설명

입력 요청 본문 {email, password}. fields()가 두 필드만 허용하고 문자열·길이(이메일 254 자, 비밀번호 128 자)를 검사합니다. 형식이 틀리면 400 BAD_INPUT 입니다. 비밀번호는 trim 하지 않고 그대로 비교합니다.

계정 확인 제공 함수 verifyCredentials()가 scrypt 해시로 비밀번호를 비교합니다. 없는 이메일이거나 비밀번호가 틀리면 null 이 되어 401 INVALID_CREDENTIALS 를 보내고, issueToken()까지 가지 않으므로 토큰이 발급되지 않습니다(T03).

발급 issueToken()은 jwt.sign()에 서버 비밀 키(SECRET)와 HS256 을 지정해 서명합니다. 클레임은 sub(사용자 ID 문자열, 예: "1"), iss(webservice-class), aud(board-api), exp(발급 시각 + TTL 초)이며 라이브러리가 iat 을 추가합니다. 비밀번호와 해시는 넣지 않습니다.

키와 유효 시간 SECRET 은 .env 의 JWT_SECRET(64 자리 16 진수)을 Buffer.from(..., "hex")로 바꾼 32 바이트입니다. 키가 비어 있거나 예시 값이거나 형식이 틀리면 auth.js 를 불러오는 순간 오류가 나서 서버가 시작되지 않습니다. TTL 은 TOKEN_TTL(기본 90 초)입니다.

출력 200 OK, {accessToken, expiresIn: 90, user: {id, name, email}} (app.js 55~59 행, T02).

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-vue/src/api.js / session, authorizationHeaders, request, login, frontend-vue/src/App.vue / logout

코드 발췌

```
// frontend-vue/src/api.js
2   let session = { accessToken: "", user: null };
8   function authorizationHeaders() {
9     return session.accessToken
10      ? { Authorization: `Bearer ${session.accessToken}` }
11      : {};
12  }
32    const data =
33      response.status === 204 ? null : await response.json().catch(() => ({}));
34    if (!response.ok) {
37      if (response.status === 401) clearSession();
50  export async function login(email, password) {
51    clearSession();
52    const data = await request("/auth/login", {
53      method: "POST",
54      body: JSON.stringify({ email, password }),
55    });
56    session = { accessToken: data.accessToken, user: data.user };
57    return data.user;
58  }
```

동작과 응답 코드 설명

로그인 응답 처리 login()은 폼의 email·password 를 JSON 으로 POST /api/auth/login 에 보내고, 200 이면 응답의 accessToken 과 user 를 session 변수에 저장합니다. 401 INVALID_CREDENTIALS 이면 request()가 오류를 던져 session 에 아무것도 저장되지 않고 '이메일 또는 비밀번호가 올바르지 않습니다.'가 표시됩니다(T03).

저장 위치 session 은 api.js 모듈 변수, 즉 이 탭의 메모리에만 있습니다. localStorage·sessionStorage 에 저장하지 않으므로 새로고침하면 로그아웃 상태가 됩니다(T16).

Authorization 헤더 request()는 모든 요청에 authorizationHeaders()의 결과를 붙입니다. 토큰이 있을 때만 'Authorization: Bearer <JWT>'를 만들고, 없으면 빈 객체라서 'Bearer undefined' 같은 헤더가 생기지 않습니다(T05 에서 헤더 첨부, T01·T11 에서 헤더 없음 확인).

응답 코드 처리 204 는 JSON 을 파싱하지 않습니다.
401(TOKEN_REQUIRED·INVALID_TOKEN·TOKEN_EXPIRED)이면 clearSession()으로 토큰과 사용자를 지우고 code 별 안내를 표시합니다. 403 FORBIDDEN 은 토큰이 유효하므로 로그인을 유지합니다. 400 은 입력값을 지우지 않고 안내하고, 404 는 상세를 비우고 목록을 다시 불러옵니다(App.vue run).

로그아웃 App.vue 의 logout()이 clearSession()을 호출하고 화면의 사용자와 입력 폼을 초기화합니다. 이후 요청에는 Authorization 헤더가 없어 /api/me 는 401 TOKEN_REQUIRED 입니다(T11). 서버에서 토큰을 무효화하는 것은 아니므로 이미 발급된 토큰은 exp 까지 유효합니다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-express/src/auth.js / currentUserId (실패 응답 생성: invalidToken)

코드 발췌

```
// backend-express/src/auth.js
38  export function currentUserId(req) {
39    const match = /^Bearer (\S+)$/i.exec(req.get("Authorization") ?? "");
40    if (!match) throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다.");
41    let payload;
42    try {
43      payload = jwt.verify(match[1], SECRET, {
44        algorithms: ["HS256"],
45        issuer: ISSUER,
46        audience: AUDIENCE,
47      });
48    } catch (error) {
50      if (error instanceof jwt.TokenExpiredError)
51        throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요.");
52      throw invalidToken();
53    }
55    if (
56      typeof payload.exp !== "number" ||
57      typeof payload.sub !== "string" ||
58      !/^[1-9]\d*$/.test(payload.sub)
59    )
60      throw invalidToken();
61    const user = findPublic(Number(payload.sub));
62    if (!user) throw invalidToken();
63    return user.id;
64  }
```

동작과 응답 코드 설명

입력 Express 요청 객체. 보호 API(GET /api/me, POST·PUT·DELETE /api/posts)는 처리를 시작할 때 이 함수를 가장 먼저 호출합니다.

① 헤더 형식(39~40 행) Authorization 이 'Bearer <토큰>' 형식이 아니면 401 TOKEN_REQUIRED 입니다(T04, T11, T17).

② 서명·알고리즘·만료·iss·aud(43~47 행) jwt.verify()에 algorithms: ["HS256"]을 지정해 다른 알고리즘(none, HS512 등)은 거부하고, 서버 키로 서명을 확인한 뒤 exp 만료와 iss·aud 값을 검사합니다. jwt.decode()는 서명을 확인하지 않고 내용만 읽기 때문에 사용하지 않았습니다.

③ 실패 구분(50~52 행) jsonwebtoken 은 서명이 맞은 뒤에 만료를 검사하므로 TokenExpiredError 는 '정상 서명의 만료 토큰'이고 401 TOKEN_EXPIRED 로 보냅니다(T10). 서명 변조, 형식 오류, iss·aud 불일치는 401 INVALID_TOKEN 입니다(서명 변조는 T09, iss·aud 불일치는 아래 추가 점검).

④ 필수 클레임(55~60 행) jwt.verify()는 exp 가 없는 토큰도 통과시키므로 exp 가 숫자인지, sub 가 양의 정수 문자열인지 직접 확인합니다.

⑤ 실제 계정(61~63 행) sub 에 해당하는 계정이 없으면 401 INVALID_TOKEN 이고, 있으면 검증된 사용자 ID(숫자)만 돌려줍니다. 라우트는 이 값만 작성자·권한 판단에 사용합니다.

출력 성공 시 사용자 ID 를 반환하고 GET /api/me 는 200 {id, name, email}을 응답합니다. 모든 401 응답에는 app.js 의 오류 처리기가 WWW-Authenticate: Bearer 헤더를 붙입니다(T04, T10, T11).

추가 점검 개발 중 로컬 키로 iss·aud 불일치, exp·sub 누락, 없는 사용자, HS512, alg=none, 다른 키로 서명한 토큰을 만들어 /api/me 에 보냈고 모두 401 INVALID_TOKEN 이었습니다. 이 점검은 AI 가 실행했습니다(docs/AI_활용기록.md).

3 4 작성자 권한 검사

파일 / 함수: backend-express/src/auth.js / assertOwner, backend-express/src/app.js / POST·PUT·DELETE /api/posts 라우트

코드 발췌

```
// backend-express/src/auth.js
67  export function assertOwner(userId, authorId) {
68    if (userId !== authorId)
69      throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");
70  }
// backend-express/src/app.js
69  app.post("/api/posts", (req, res) => {
70    const userId = currentUserId(req);
71    const p = store.create(
72      fields(req.body, { title: 100, content: 5000 }),
73      userId,
74    );
77  app.put("/api/posts/:id", (req, res) => {
78    const userId = currentUserId(req),
79      id = postId(req),
80      p = getPost(id);
81    assertOwner(userId, p.authorId);
82    res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
```

동작과 응답 코드 설명

비교 대상 currentUserId()가 토큰에서 검증한 현재 사용자 ID 와, getPost()로 DB 에서 다시 읽은 글의 authorId 를 비교합니다. 요청 본문이나 화면의 값은 비교에 쓰지 않습니다. DELETE(84~90 행)도 같은 순서로 assertOwner()를 호출합니다.

판단 순서 PUT·DELETE 는 인증(currentUserId, 401) → ID 형식(postId, 400) → 글 존재(getPost, 404) → 작성자 권한(assertOwner, 403) → 입력 검사(fields, 400) → 저장 순서입니다. 그래서 Bob 이 Alice 글에 공백 제목을 보내면 403, Alice 가 자기 글에 보내면 400 입니다(X_order.png).

401 과 403 의 차이 401 은 누구인지 확인하지 못한 경우(토큰 없음·변조·만료)이고, 403 은 누구인지는 확인했지만 그 글을 바꿀 권한이 없는 경우입니다. 그래서 화면도 401 에서는 로그인을 해제하고 403 에서는 로그인을 유지합니다.

작성자 위조 방지 글 등록 시 store.create()에 넘기는 작성자는 요청 본문이 아니라 currentUserId()가 돌려준 ID 입니다. fields()는 title·content 외의 필드를 허용하지 않아 authorId·userId 를 넣으면 400 BAD_INPUT 입니다(T12).

결과 Bob 의 정상 토큰으로 Alice 글을 수정·삭제하면 403 FORBIDDEN 이고 재조회 시 글이 그대로입니다(T07, T08). 본인 수정은 200 이며 authorId 와 createdAt 이 유지되고, 본인 삭제는 204 입니다(T06, T17). 프론트의 canEdit()은 버튼 표시용이며 서버 검사는 이와 관계없이 항상 수행됩니다.

테스트 결과

실행 환경: macOS 15.6.1 / Node.js v24.15.0, Python 3.10.2 / Chrome 154 / 백엔드 npm start, 프론트 npm run build → npm run preview

로컬 배포 확인: npm ci 로 의존성 설치 후 프론트 빌드 성공(E01_build.png), backend-express 에서 npm start, http://127.0.0.1:4173 에서 GET /api/posts 200(E02_preview.png). 새 폴더 clone 재현도 README 절차로 확인(C01~C04)

테스트 일시 및 코드 버전: 2026-10-28 15:20~17:10 (KST) / 커밋 e67b522

토큰 TTL: 일반 테스트 90 초 / 만료 테스트 3 초 (TOKEN_TTL=3 npm start)

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인으로 목록·글 20 상세 조회(화면)	200, 공개 조회	200 / 해당 없음, Authorization 헤더 없음	PASS	T01_public, T01_public_reqheaders
T02	Alice 화면 로그인	200, JWT·사용자 정보, 로그인 표시	200 / 해당 없음, 'Alice 님 로그인 중'	PASS	T02_login, T02_login_headers
T03	틀린 비밀번호, 없는 이메일 로그인(화면·curl)	401 INVALID_CREDENTIALS, 토큰 미발급	401 / INVALID_CREDENTIALS ×2, accessToken 없음	PASS	T03_wrong_pw, T03_unknown_email, T03_curl
T04	토큰 없이 GET /me, POST /posts(curl)	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED ×2, WWW-Authenticate: Bearer	PASS	T04_no_token
T05	Alice 글 등록 후 내 정보 조회(화면)	201, authorId 1, /me Alice	201 / 해당 없음(Location /api/posts/20, authorId 1), /me 200	PASS	T05_create_201_a·_b, T05_create_preview, T05_me
T06	Alice 본인 글 21 수정·삭제(화면)	200 및 204	200, 204 / 해당 없음, 삭제 응답 본문 없음	PASS	T06_update_200, T06_delete_204, T06_delete_empty
T07	Bob 토큰으로 글 20 PUT(curl)	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN, 재조회 원문 유지	PASS	T07_bob_put_403, T07_ui_hidden_a·_b
T08	Bob 토큰으로 글 20 DELETE(curl)	403 FORBIDDEN, 글 유지	403 / FORBIDDEN, 재조회 글 유지	PASS	T08_bob_delete_403
T09	서명 변조 토큰으로 /me·POST·PUT·DELETE(curl)	401 INVALID_TOKEN, 변경 없음	401 / INVALID_TOKEN ×4, 변경 없음	PASS	T09_tampered

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T10	TTL 3 초, 5 초 뒤 /me·POST·PUT·DELETE(curl)와 화면 /me	401 TOKEN_EXPIRED, 로그인 해제·안내	401 / TOKEN_EXPIRED ×5, 화면 로그아웃 상태·만료 안내	PASS	T10_server_ttl3, T10_expired_curl, T10_expired_ui_a·_b
T11	로그인 → 로그아웃 → 내 정보 조회(화면)	헤더 없음, 401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED, Authorization 헤더 없음	PASS	T11_logout, T11_logout_reqheaders
T12	Bob 등록에 authorId:1, userId:1 각각 추가(curl)	400 BAD_INPUT, 위조 글 없음	400 / BAD_INPUT ×2, 위조 글 없음	PASS	T12_forge
T13	빈 제목·공백 본문·제목 101 자·내용 5001 자(curl)	400 BAD_INPUT, 변경 없음	400 / BAD_INPUT ×4, 변경 없음	PASS	T13_validation
T14	없는 ID 999999 조회·수정·삭제, /posts/abc(curl)	없는 글 404, 잘못된 ID 400	404 / NOT_FOUND ×3, 400 / BAD_INPUT	PASS	T14_404_400
T15	백엔드 Ctrl+C 종료 후 npm start 재시작	글·작성자·내용 유지	200 / 해당 없음, 재시작 전후 글 20 동일	PASS	T15_restart_server·_curl·_ui
T16	Alice 로그인 후 새로고침(화면)	글 유지, 로그인 해제	해당 없음, 로그아웃 상태·글 20 유지	PASS	T16_before, T16_refresh
T17	무토큰 PUT·DELETE, Bob 본인 작성·수정·삭제(curl)	무토큰 401, Bob 본인 성공	401 / TOKEN_REQUIRED ×2, 201·200·204	PASS	T17_noauth_401, T17_bob_own

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 / 해당 없음 (목록·상세, 로그인, /me, 본인 수정)	T01, T02, T05_me, T06_update_200, T17
새 글 등록	**201 및 Location**	201, Location: /api/posts/20(Alice)· /api/posts/22(Bob)	T05_create_201_a, T17_bob_own
본인 글 삭제	**204 및 빈 본문**	204, 응답 본문 없음	T06_delete_204, T06_delete_empty, T17
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT (빈 제목·공백 본문·길이 초과, authorId·userId, /posts/abc)	T12, T13, T14
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS (틀린 비밀번호, 없는 이메일)	T03
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED (헤더 없음, Basic 방식, Bearer 만, 접두어 없는 토큰)	T04, T11, T17, C05
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN (서명 변조, /me·POST·PUT·DELETE)	T09
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED (TTL 3 초, /me·POST·PUT·DELETE, 화면 /me)	T10
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN (Bob → Alice 글 PUT·DELETE, 공백 제목 PUT)	T07, T08, X_order

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND (999999 GET·PUT·DELETE)	T14

WWW-Authenticate: Bearer 토큰 없는 /me·POST(T04), 로그아웃 후 /me(T11), 만료 토큰 /me(T10)의 응답 헤더에서 WWW-Authenticate: Bearer 를 확인했습니다. 로그인 실패 401(T03)에도 같은 헤더가 붙습니다.

401 과 403 의 화면 차이 401(토큰 없음·만료)이면 메모리의 토큰과 사용자가 지워져 '로그아웃 상태'로 바뀌고 '로그인이 필요합니다', '로그인 시간이 만료되었습니다' 안내가 표시됩니다(T10, T11). 403 은 api.js request()에서 401 일 때만 세션을 지우도록 했기 때문에 로그인 상태가 유지되고 권한 안내만 표시됩니다. 수동 테스트에서는 Bob 화면에 Alice 글의 수정·삭제 버튼이 보이지 않아(T07_ui_hidden) 403 은 curl 로 직접 요청해 확인했습니다.

실패 후 데이터 불변 T07·T08(Bob 수정·삭제 403), T09(변조 토큰 4 건), T10(만료 토큰), T12(위조 필드), T13(입력 오류), T17(무토큰) 뒤에 매번 GET /api/posts/20 또는 목록을 다시 조회했고, 글 20 의 제목·내용·updatedAt 이 바뀌지 않았으며 새 글도 생기지 않았습니다.

토큰 없이 / 정상 토큰으로 없는 글 수정 토큰 없이 999999 수정은 401 TOKEN_REQUIRED, Alice 의 정상 토큰으로 같은 글 수정은 404 NOT_FOUND 였습니다(X_order.png, T14_404_400.png). 서버가 인증을 글 존재 확인보다 먼저 하기 때문입니다.

테스트 증거

로그인과 토큰 전송

증거 1: Alice 로그인 성공(T02)과 토큰이 첨부된 글 등록 요청(T05). 토큰은 서명 부분이 보이지 않게 잘라서 캡처했습니다.

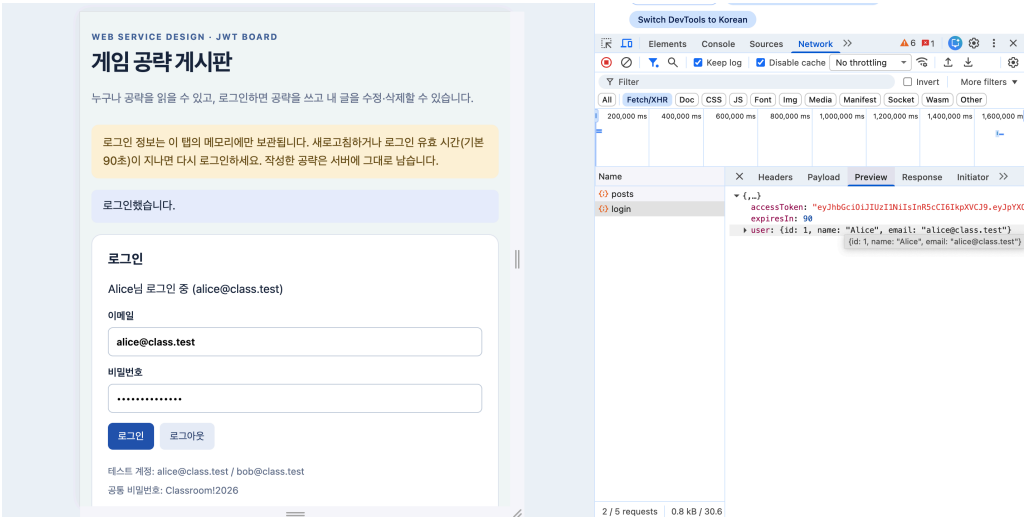


그림 1. T02 Alice 로그인 200, 응답의 accessToken·expiresIn·user 와 화면 로그인 표시 (T02_login.png)

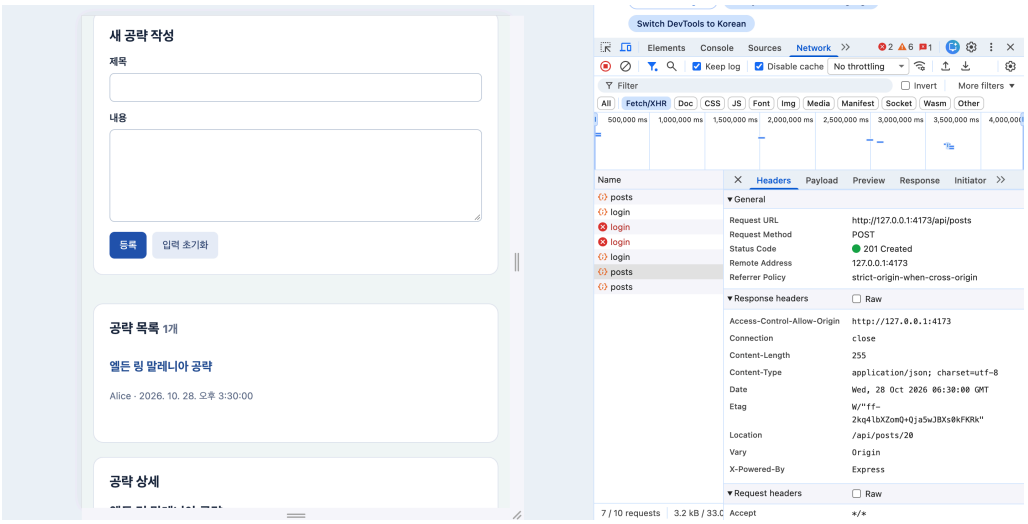


그림 2. T05 글 등록 201 Created, Location: /api/posts/20 (T05_create_201_a.png)

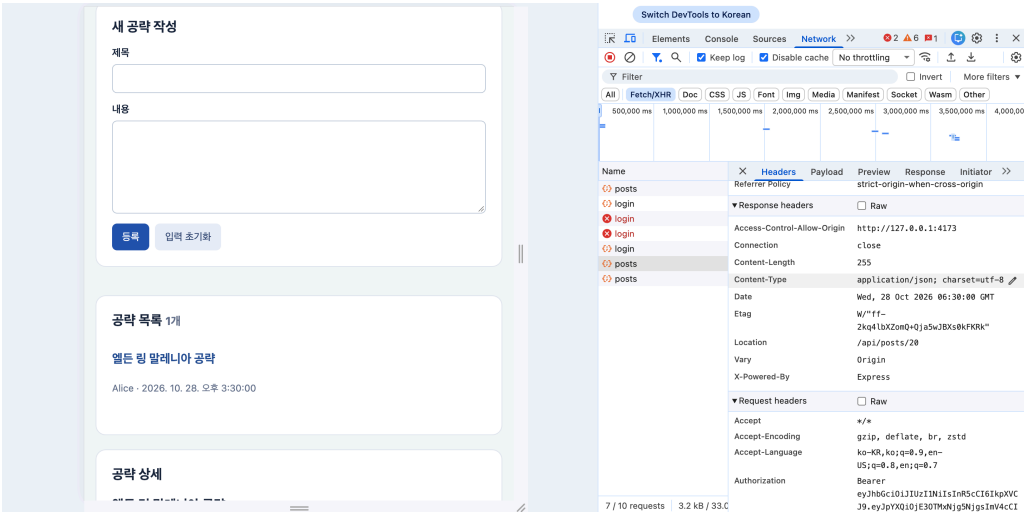


그림 3. T05 글 등록 요청의 Authorization: Bearer 헤더 (T05_create_201_b.png)

다른 작성자의 수정과 삭제

증거 2: Bob 의 정상 토큰으로 Alice 글(20) 수정·삭제를 직접 요청한 서버 응답(T07, T08)과 Bob 화면

```
 PID=20
Bb=$(tok bob); peek "$Bb"
curl -s -i -X PUT $B/posts/$PID -H "$J" -H "Authorizat
ion: Bearer $Bb" -d '{"title":"Bob이 바꿈","content":"
위조"}'; echo; echo
curl -s $B/posts/$PID; echo
eyJhbGciOi…(가림)  길이 199
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 95
ETag: W/"5f-1EXFvGmBiuvfByYKDskXBCi5WiA"
Date: Wed, 28 Oct 2026 07:14:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"code":"FORBIDDEN","message":"본인이 작성한 글만 수정
·삭제할 수 있습니다."}

{"id":20,"title":"엘든 링 말레니아 공략","content":"2
페이즈 개화 패턴은 멀리 달려서 피하세요. 출혈 무기 추
천.","authorId":1,"authorName":"Alice","createdAt":"20
26-10-28T06:30:00.982Z","updatedAt":"2026-10-28T06:30:
00.982Z"}
```

그림 4. T07 Bob → Alice 글 PUT 403 FORBIDDEN, 재조회 원문 유지 (T07_bob_put_403.png)

```
 Bb=$(tok bob)
curl -s -i -X DELETE $B/posts/$PID -H "Authorization:
Bearer $Bb"; echo; echo
curl -s $B/posts/$PID; echo
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 95
ETag: W/"5f-1EXFvGmBiuvfByYKDskXBCi5WiA"
Date: Wed, 28 Oct 2026 07:18:01 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"code":"FORBIDDEN","message":"본인이 작성한 글만 수정
·삭제할 수 있습니다."}

{"id":20,"title":"엘든 링 말레니아 공략","content":"2
페이즈 개화 패턴은 멀리 달려서 피하세요. 출혈 무기 추
천.","authorId":1,"authorName":"Alice","createdAt":"20
26-10-28T06:30:00.982Z","updatedAt":"2026-10-28T06:30:
00.982Z"}
```

그림 5. T08 Bob → Alice 글 DELETE 403 FORBIDDEN, 재조회 글 유지 (T08_bob_delete_403.png)

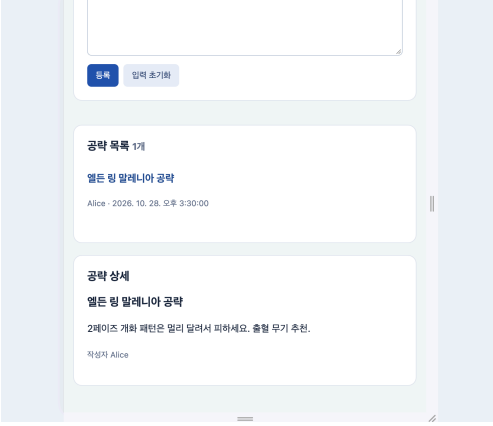


그림 6. Bob 로그인 화면에서 Alice 글에 수정·삭제 버튼 없음 (T07_ui_hidden_b.png)

토큰 변조와 만료

증거 3: 서명 변조 토큰(T09)과 TTL 3 초 만료 토큰(T10)에 대한 서버 응답과 화면 처리

```
● A=$(tok alice); BAD=$(tamper "$A"); peek "$A"; peek "
  $BAD"
  curl -s -w "  → %{http_code}\n" $B/me -H "Authorizatio
  n: Bearer $BAD"
  curl -s -w "  → %{http_code}\n" -X POST $B/posts -H "$
  J" -H "Authorization: Bearer $BAD" -d '{"title":"변조"
  ,"content":"변조"}'
  curl -s -w "  → %{http_code}\n" -X PUT $B/posts/$PID -
  H "$J" -H "Authorization: Bearer $BAD" -d '{"title":"
  변조","content":"변조"}'
  curl -s -w "  → %{http_code}\n" -X DELETE $B/posts/$PI
  D -H "Authorization: Bearer $BAD"
  curl -s $B/posts; echo
  eyJhbGciOi…(가림)  길이 199
  eyJhbGciOi…(가림)  길이 199
  {"code":"INVALID_TOKEN","message":"유효하지 않은 토큰
  입니다. 다시 로그인하세요."}  → 401
  {"code":"INVALID_TOKEN","message":"유효하지 않은 토큰
  입니다. 다시 로그인하세요."}  → 401
  {"code":"INVALID_TOKEN","message":"유효하지 않은 토큰
  입니다. 다시 로그인하세요."}  → 401
  {"code":"INVALID_TOKEN","message":"유효하지 않은 토큰
  입니다. 다시 로그인하세요."}  → 401
  [{"id":20,"title":"엘든 링 말레니아 공략","content":"2
  페이즈 개화 패턴은 멀리 달려서 피하세요. 출혈 무기 추
  천.","authorId":1,"authorName":"Alice","createdAt":"20
  26-10-28T06:30:00.982Z","updatedAt":"2026-10-28T06:30:
  00.982Z"}]
```

그림 7. T09 서명 변조 토큰으로 /me·POST·PUT·DELETE 모두 401 INVALID_TOKEN (T09_tampered.png)

```
● A=$(tok alice); peek "$A"; echo "토큰 발급, 5초 대기.
  .."; sleep 5
  curl -s -i $B/me -H "Authorization: Bearer $A" | grep
  -iE "^HTTP|^www-auth|^\{"
  curl -s -w "  → %{http_code}  (POST)\n" -X POST $B/pos
  ts -H "$J" -H "Authorization: Bearer $A" -d '{"title":
  "만료","content":"만료"}'
  curl -s -w "  → %{http_code}  (PUT)\n" -X PUT $B/posts
  /$PID -H "$J" -H "Authorization: Bearer $A" -d '{"titl
  e":"만료","content":"만료"}'
  curl -s -w "  → %{http_code}  (DELETE)\n" -X DELETE $B
  /posts/$PID -H "Authorization: Bearer $A"
  curl -s $B/posts; echo

  eyJhbGciOi…(가림)  길이 199
  토큰 발급, 5초 대기...
  HTTP/1.1 401 Unauthorized
  WWW-Authenticate: Bearer
  {"code":"TOKEN_EXPIRED","message":"로그인 시간이 만료
  되었습니다. 다시 로그인하세요."}
  {"code":"TOKEN_EXPIRED","message":"로그인 시간이 만료
  되었습니다. 다시 로그인하세요."}  → 401  (POST)
  {"code":"TOKEN_EXPIRED","message":"로그인 시간이 만료
  되었습니다. 다시 로그인하세요."}  → 401  (PUT)
  {"code":"TOKEN_EXPIRED","message":"로그인 시간이 만료
  되었습니다. 다시 로그인하세요."}  → 401  (DELETE)
  [{"id":20,"title":"엘든 링 말레니아 공략","content":"2
  페이즈 개화 패턴은 멀리 달려서 피하세요. 출혈 무기 추
  천.","authorId":1,"authorName":"Alice","createdAt":"20
  26-10-28T06:30:00.982Z","updatedAt":"2026-10-28T06:30:
  00.982Z"}]
```

그림 8. T10 5 초 뒤 /me·POST·PUT·DELETE 모두 401 TOKEN_EXPIRED (T10_expired_curl.png)

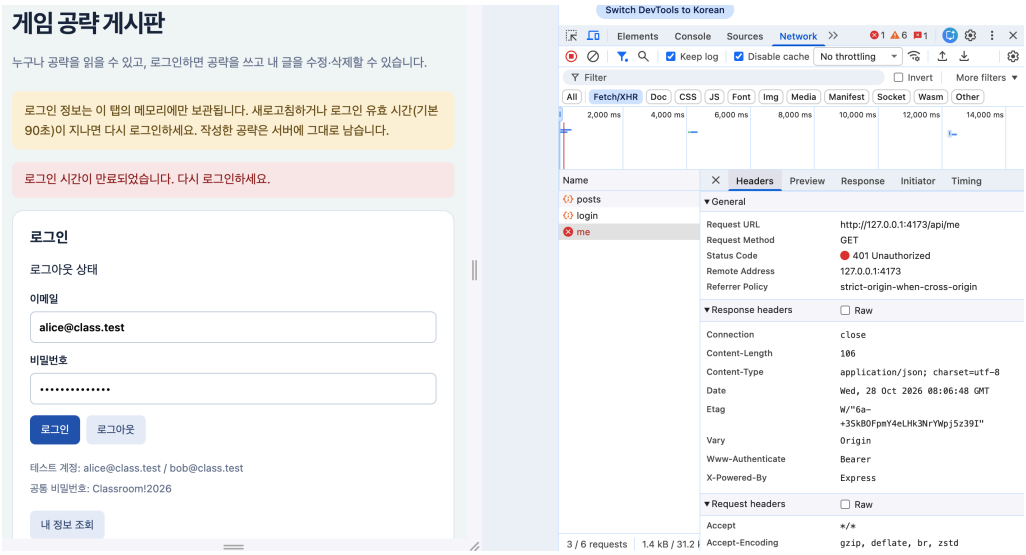


그림 9. T10 화면 만료 안내와 로그아웃 상태 전환, /me 401 (T10_expired_ui_a.png)

자동 검사와 로컬 배포 증거

자동 검사 저장소 루트에서 백엔드를 켠 상태로 python3 tools/check_api.py --mode final 실행 → All 40 checks passed(E03_check_final.png). TOKEN_TTL=3 npm start 로 재시작한 뒤 --mode final --expiry 실행 → All 48 checks passed(E04_check_expiry.png).

제공 검사 도구는 핵심 API 동작만 자동으로 확인하므로, 화면 처리·로그아웃·새로고침·서버 재시작은 위의 수동 테스트로 따로 확인했습니다.

빌드 성공 로그, 4173 번 API 연결, 서버 재시작 전후 데이터 유지 화면은 아래와 같습니다. 전체 캡처는 docs/evidence/(41 장), 결과표는 docs/evidence/테스트기록.md 에 있습니다.

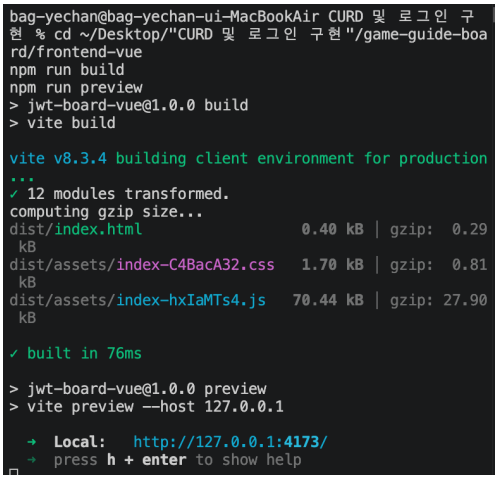


그림 10. npm run build 성공과 preview 주소 (E01_build.png)

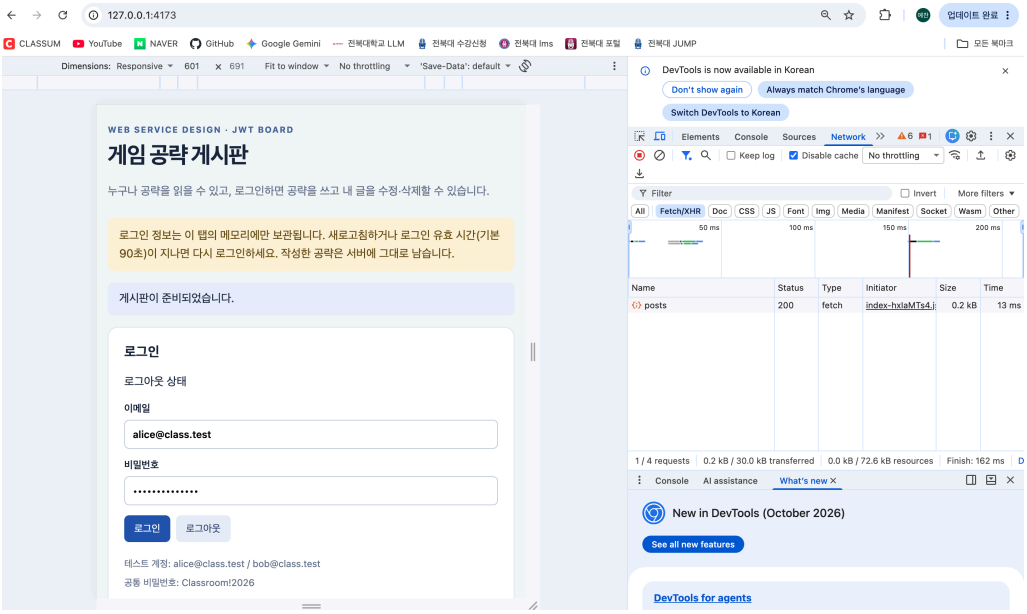


그림 11. 4173 번 접속과 GET /api/posts 200 (E02_preview.png)

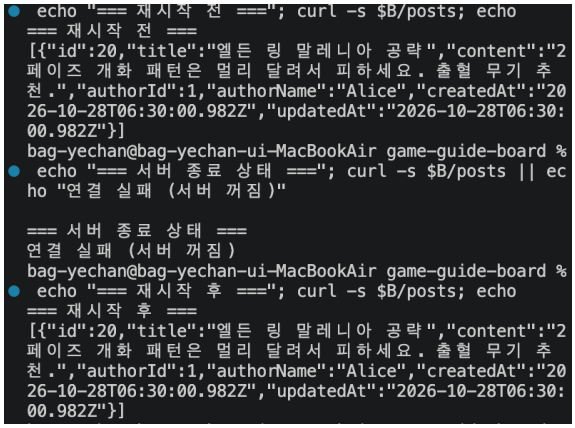


그림 12. T15 서버 종료·재시작 전후 글 20 동일 (T15_restart_curl.png)

AI 활용과 참고 자료

AI 사용 여부: 사용 (Claude Code, 모델 Claude Opus 5.5). 단계별 상세 기록은 docs/AI_활용기록.md 에 있습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude Code (VS Code) / Claude Opus 5.5	학습·설계	과제 문서 요약, 작업 순서 계획, 요청 흐름 설명	참고
Claude Code (VS Code) / Claude Opus 5.5	생성	backend-express/src/auth.js (issueToken, currentUserId, assertOwner, 키·TTL 검사)	그대로 반영
Claude Code (VS Code) / Claude Opus 5.5	생성	frontend-vue/src/api.js (login, authorizationHeaders, request, canEdit), App.vue, style.css, index.html	그대로 반영
Claude Code (VS Code) / Claude Opus 5.5	테스트	자동 검사·추가 클레임 점검 실행, T01~T17 수동 테스트 절차와 curl 명령 안내 (테스트 수행과 캡처는 본인)	참고
Claude Code (VS Code) / Claude Opus 5.5	문장정리	README.md, docs/evidence/테스트기록.md	그대로 반영(본인 확인)
Claude Code (VS Code) / Claude Opus 5.5	문장정리	본 보고서 초안 (docs/202219903_박예찬_최종보고서.pdf)	수정 후 반영

대표 사례

사례 1. 백엔드 JWT 발급·검증·작성자 권한 코드 생성

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10-28

요청한 내용: auth.js 의 BE-01~03(JWT 발급, 토큰 검증, 작성자 권한)을 공지와 API 명세대로 구현해 달라고 요청

AI 가 제안한 내용: jwt.sign()으로 HS256·sub·iss·aud·exp 발급, jwt.verify()에 algorithms·issuer·audience 를 지정해 검증, TokenExpiredError 는 TOKEN_EXPIRED 로 구분, exp·sub 와 실제 계정을 직접 확인, userId 와 authorId 비교로 403. JWT_SECRET 이 예시 값이거나 64 자리 16 진수가 아니면 서버 시작 거부

실제 반영 위치: backend-express/src/auth.js 전체 / 보고서 3.1, 3.3, 3.4

내가 확인하고 수정한 내용: 코드를 읽고 검증 순서와 401·403 구분을 확인했습니다. 공지의 JWT 구현 기준(알고리즘 고정, exp·iss·aud·sub 검증, 예시 키 거부)과 비교해 빠진 항목이 없었고 테스트가 모두 통과해 수정하지 않고 그대로 반영했습니다.

검증 방법과 결과: T02~T04, T07~T10, T12~T14, T17 을 직접 수행해 모두 PASS(예: Bob → Alice 글 PUT 403 FORBIDDEN, 변조 토큰 401 INVALID_TOKEN, TTL 3 초 토큰 401 TOKEN_EXPIRED). 자동 검사 final 40 개, expiry 48 개 통과

사례 2. 프론트 로그인 연결과 응답 코드별 처리

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10-28

요청한 내용: FE-01~04(로그인 상태, Bearer 헤더, 응답 코드별 처리, 작성자별 버튼)를 구현해 달라고 요청

AI 가 제안한 내용: 토큰을 api.js 모듈 변수에만 저장, 토큰이 있을 때만 Authorization 헤더 생성, 401 이면 세션 해제·403 이면 유지, code 별 안내 문구, 204 는 JSON 파싱 생략, canEdit 은 사용자 ID 와 authorId 비교. App.vue 에 비로그인 글쓰기 안내, 404 시 목록 갱신, 데모 배너를 게임 공략 안내로 교체

실제 반영 위치: frontend-vue/src/api.js login·authorizationHeaders·request·canEdit, App.vue run·save·logout / 보고서 3.2

내가 확인하고 수정한 내용: localStorage 를 쓰지 않아 새로고침 시 로그아웃되는 것이 공지의 요구(토큰은 메모리 보관)와 맞는지, 403 에서 로그아웃하지 않는 것이 FE-03 기준과 맞는지 확인했습니다. 요구사항과 일치해 수정하지 않았습니다.

검증 방법과 결과: Network 에서 T05 요청의 Authorization: Bearer 첨부, T01·T11 요청의 Authorization 없음, T10 만료 시 화면 로그아웃 전환, T16 새로고침 후 로그인 해제·글 유지를 직접 확인(모두 PASS)

사례 3. 수동 테스트 진행 안내

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10-28

요청한 내용: 테스트 가이드가 너무 길어서 한 단계씩 알려 달라고 요청

AI 가 제안한 내용: T01~T17 을 화면·curl·서버 재시작 순서로 나누어 한 단계씩 안내하고, 토큰 원문을 출력하지 않는 curl 보조 함수(tok, tamper)와 캡처 파일 이름을 제시

실제 반영 위치: docs/evidence/ 캡처 41 장, docs/evidence/테스트기록.md / 보고서 4 장

내가 확인하고 수정한 내용: 모든 테스트를 직접 브라우저와 터미널에서 수행하고 캡처했습니다. GET /api/posts 가 304 로 표시되는 것을 발견해 Disable cache 를 켜고 200 을 다시 확인했고, 한 화면에 안 들어가는 캡처는 두 장으로 나누었으며, T11 에서는 Authorization 헤더가 없다는 것이 보이도록 요청 헤더 전체를 다시 캡처했습니다.

검증 방법과 결과: T01~T17 모두 PASS. 기록표의 값은 캡처에 찍힌 상태 코드·code·헤더와 일치

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
제공 시작 템플릿(JWT-Assignment)과 과제 문서 docs/01~07 (수업 자료)	app.js·store.js·users.js·errors.js, frontend-vue 구조, tools/check_api.py	제공 코드 유지, 폴더 구조 동일. API 명세·테스트 시나리오를 구현·검증 기준으로 사용
jsonwebtoken (auth0/node-jsonwebtoken) https://github.com/auth0/node-jsonwebtoken	backend-express/src/auth.js / issueToken, currentUserId	jwt.sign·jwt.verify 옵션과 TokenExpiredError 사용. 템플릿에 포함된 의존성(MIT 라이선스)

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
Vite 공식 문서 https://vite.dev/guide/	frontend-vue/vite.config.js(제공 코드), 실행 방법	npm run build·preview 와 /api 프록시 동작 이해

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorId 를 믿으면 로그인한 사용자가 본문에 다른 사람의 ID(예: Bob 이 authorId: 1)를 넣어 Alice 가 쓴 것처럼 글을 만들 수 있습니다. 내 코드는 글 등록 시 작성자를 currentUserId()가 토큰에서 검증한 ID 로만 정하고(app.js 70~73 행), fields()가 title·content 외 필드를 400 BAD_INPUT 으로 거부합니다. Bob 토큰에 authorId:1, userId:1 을 각각 넣었을 때 모두 400 이었고 위조 글은 생기지 않았습니다(T12).

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼 숨김은 화면에서만 막는 것이라 curl 이나 API 클라이언트로 PUT·DELETE 를 직접 보내면 우회됩니다. 서버에서는 auth.js 의 assertOwner()가 검증된 사용자 ID 와 DB 의 authorId 를 비교해 다르면 403 을 반환합니다. 실제로 Bob 화면에는 버튼이 없었지만, curl 로 직접 보낸 수정·삭제 요청도 403 으로 거부되고 원문이 유지됐습니다(T07, T08).

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 JWT 의 payload 를 base64 로 풀어 읽기만 하고 서명을 확인하지 않아서, 누군가 sub 를 바꾼 토큰도 그대로 믿게 됩니다. verify 는 서버 키로 서명을 다시 계산해 비교하고 만료도 검사합니다. 내 코드는 jwt.verify()에 HS256 만 허용하고 iss·aud 를 지정해 서명·알고리즘·만료·발급자·대상을 검사하며, 추가로 exp 존재, sub 형식, 실제 계정 존재를 확인합니다(auth.js currentUserId).

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 이 탭 메모리의 토큰과 사용자 정보를 지워 이후 요청에 Authorization 헤더가 붙지 않게 합니다(T11). 토큰 만료는 서버가 exp 가 지난 토큰을 401 TOKEN_EXPIRED 로 거부하는 것이고, 화면은 이 응답을 받으면 로그인 상태를 해제합니다(T10). 새로고침은 메모리가 초기화되어 로그인 상태만 사라지고 게시글은 DB 에 남습니다(T16). 서버가 토큰 목록을 관리하지 않으므로, 이미 복사된 토큰은 로그아웃과 관계없이 exp(기본 90 초)가 지날 때까지 유효합니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

빌드본을 열었을 때 Network 에서 GET /api/posts 의 상태가 명세의 200 이 아니라 304 로 표시됐습니다. AI 에게 물어 원인을 확인해 보니, 같은 페이지를 전에 열었기 때문에 브라우저가 ETag 로 캐시를 재검증하고 서버가 304 Not Modified 로 답한 것이었습니다. 코드는 바꾸지 않고 개발자 도구 Network 의 Disable cache 를 켠 뒤 새로고침해서 서버가 실제로 200 을 보내는 것을 다시 확인했고(E02_preview.png), 이후 모든 화면 테스트를 캐시를 끈 상태로 진행했습니다.

현재 남은 문제와 다음에 개선할 점: 알려진 기능 오류는 없습니다. 로그인 상태가 메모리에만 있어 새로고침하면 다시 로그인해야 하고, 로그아웃해도 이미 발급된 토큰은 만료 전까지 유효합니다. 다음에는 Refresh Token 이나 서버 측 토큰 폐기 목록을 적용해 보고, 수동으로 한 화면 테스트 일부를 자동화하고 싶습니다.

제출 확인: [x] 실제 코드와 설명 일치 [x] 테스트 직접 수행 [x] AI 활용 및 출처 기록 [x] 비밀 키와 토큰 제거

GitHub 확인: [x] 루트 README 작성 [x] 보고서 PDF 와 증거 업로드 [x] README 문서 링크 확인 [x] 교수자 접근 가능 [x] 저장소 URL 과 커밋 SHA 제출